

基于微服务的AI应用项目-Chat2Excel问题汇总

通用问题

1. 为什么做 Chat2Excel 这个项目？

- 回答 1：（项目体验）

在 Chat2Excel 项目中，我将 JavaWeb、微服务、大模型集成等技术进行了系统性整合运用。从项目架构设计、微服务拆分（用户 / 文件 / AI/MCP 服务），到前后端接口定义、中间件部署（MySQL/Redis/Nacos），再到最终的开发测试与云服务器部署，完整经历了企业级项目的全流程，让我对微服务架构和 AI 应用开发有了更深入的实际认知。

- 回答 2：（个人兴趣）

我对“大模型 + 办公场景”的融合应用非常感兴趣，日常发现业务人员处理 Excel 数据时存在手动分析效率低、SQL 门槛高的痛点。而 Chat2Excel 通过自然语言交互解决了这一问题，且项目可集成多模型、支持流式响应等前沿技术，能让我深入探索大模型与传统业务系统的结合逻辑。

- 回答 3：（学校课程设计）

我们软件工程课程要求实现一个“AI + 数据处理”相关的 Web 系统，我结合课程所学的微服务、数据库设计知识，参考开源社区的技术方案，最终完成了 Chat2Excel 项目，既满足了课程对系统架构和功能完整性的要求，也实践了 AI 集成的前沿技术。

2. 你的项目为什么跟我前面面试同学的项目一样？

- 回答 1：（项目体验）

我在 Chat2Excel 项目开发中，主要参考了 Gitee 上相关的微服务 + AI 集成开源方案，可能上一位同学也借鉴了类似的开源项目思路。但这个项目除了借鉴常规的架构拆分流程，我还进行了功能扩展，比如在 AI 图表生成模块中，新增了“图表数据导出 Excel”功能，用户生成图表后可直接下载包含原始数据的 Excel 文件，解决了仅展示图表无法复用数据的痛点；同时优化了多 Sheet 数据关联查询逻辑，通过缓存 Sheet 关联字段映射关系，提升了跨 Sheet 分析的响应速度。

- 回答 2：（个人兴趣）

Chat2Excel 聚焦的“大模型 + Excel 数据处理”是当下热门的应用场景，可能很多同学都关注到这个方向并进行实践。但我在项目中做了性能优化，针对大文件 Excel 解析容易内存溢出的问题，采用了分块读取 + 内存释放机制，将单文件解析的内存占用从原来的数百 MB 降至几十 MB，支持更大体积文件的快速解析；还优化了 JWT Token 的缓存策略，通过 Redis 哈希结构分类存储用户 Token 和权限信息，减少了鉴权时的查询耗时。

- 回答 3：（学校课程设计）

这个项目确实是我们学校课程设计的推荐方向，要求实现 “AI + 数据处理” 相关功能。我在开发时，除了参考学校提供的基础框架，还在网上查找了类似的开源项目和技术博客补充细节，可能上一位同学也参考了同类资源。但我做了代码优化，比如封装了通用的 Excel 工具类，统一处理表头标准化、数据类型转换等逻辑，减少了重复代码；同时新增了接口参数校验注解和全局异常处理，让系统在面对非法输入时能返回更友好的提示，提升了鲁棒性。

3. 介绍一下 Chat2Excel 项目的核心功能和整体架构？

- 核心功能：
 - 用户认证（验证码 / 密码登录）
 - Excel 上传 / 预览 / 下载 / 删除、
 - 自然语言数据分析（查询 / 修改 / 图表生成）多模型切换、历史会话管理。
- 整体架构：
 - 浏览器层（前端交互界面）
 - 网关层（Nginx 反向代理 + Spring Cloud Gateway 路由鉴权）
 - 业务服务层（用户 / 文件 / AI/MCP/ 公共服务）
 - 注册发现层（Nacos）
 - 数据存储层（MySQL+Redis + 阿里云 OSS）
 - 基础设施层（Docker 容器化部署）。

4. 你在项目中使用了哪些技术栈？

- 核心技术栈：
 - Spring Boot/Spring Cloud（微服务架构基石）
 - Spring AI（简化大模型集成）
 - MySQL+MyBatis-Plus（结构化数据存储）
 - Redis（缓存 + 分布式锁）
 - Nacos（服务注册 + 配置中心）
 - JWT（鉴权）
 - Docker（环境一致性）
 - 阿里云 OSS（文件存储）
 - EasyExcel（Excel 解析）。
- 选择理由：
 - Spring 生态成熟，适配企业级开发需求；
 - Spring AI 可快速集成多模型，降低开发成本；

- Redis 支持高并发场景下的缓存和分布式协同；
- OSS 解决文件存储扩容问题；
- Docker 确保部署环境一致性，避免 “开发环境能跑，生产环境报错”。

5. 你从这个项目中学到了什么？

- 项目成果：支持 Excel (xlsx/xls) 上传解析、自然语言转 SQL 查询、多类型图表生成（柱状图 / 折线图）数据修改与 Excel 导出，响应延迟低，支持 50MB 以内大文件处理，部署在阿里云服务器可稳定访问。
- 学到的内容：深入理解微服务拆分与服务通信机制；掌握大模型集成（多模型适配、流式响应、Tool Calling）核心逻辑；熟练运用 Redis 分布式锁、JWT 鉴权、OSS 文件存储等企业级组件；提升了问题排查（如索引优化、流式响应超时）和项目全流程（设计 - 开发 - 测试 - 部署）管理能力。

6. 整个项目做了多长时间？

项目耗时 2 个月，平衡思路如下：

- 明确核心范围：优先实现 “上传 - 解析 - 查询 - 图表” 核心流程，次要功能（如高级权限管理）后续迭代；
- 优先级排序：先完成基础模块（用户认证、文件存储），再开发 AI 集成和流式响应等高复杂度功能；
- 预留缓冲时间：因抱着学习心态，未制定刚性时间节点，将问题解决（如索引优化、多模型适配）纳入学习过程，确保技术实现质量。

7. 开发过程中遇到的最大挑战是什么？

- 挑战 1：大文件 Excel 解析内存溢出。识别：测试时上传 较大文件时出现 OOM 异常；解决：采用 EasyExcel 流式读取，分批次解析数据，避免一次性加载整个文件到内存，同时限制单文件最大 50MB。
- 挑战 2：多模型切换时的兼容性问题。识别：DashScope 和 DeepSeek 的 API 参数、响应格式存在差异；解决：基于 Spring AI 封装统一接口，通过工厂模式适配不同模型的调用逻辑，配置中心动态切换默认模型。
- 挑战 3：流式响应（SSE）超时问题。识别：AI 处理复杂查询时，长连接易断开；解决：设置 300 秒超时时间，通过 SseEmitter 分阶段推送处理进度，确保连接稳定。

8. 你是如何测试和部署这个项目的？

- 测试：
 - 针对核心功能设计测试用例，
 - 如 Excel 上传（测试不同格式 / 大小文件）

- AI 查询（测试不同自然语言指令转 SQL 准确性）
- 权限校验（测试无效 Token / 白名单接口）；
- 采用手动测试 + 接口自动化测试结合，验证功能可用性和边界情况。
- 部署：
 - 构建自动化：通过 Maven 编译代码，生成可执行 Jar 包；
 - 中间件部署：用 Docker Compose 启动 MySQL、Redis、Nacos、Nginx，确保环境一致性；
 - 服务部署：将 Jar 包上传至阿里云服务器，通过 nohup 后台启动，配置日志输出；
 - 配置：Nginx 反向代理转发请求，Nacos 管理服务注册和配置，确保服务协同。

9. 项目中采取了哪些安全措施来确保数据和接口安全？

- 鉴权机制：基于 JWT+Redis 实现身份校验，Token 过期自动失效，支持刷新机制；
- 网关防护：统一拦截请求，白名单管理公开接口（登录 / 验证码），非白名单接口需鉴权；
- 文件安全：校验文件格式（仅支持 xlsx/xls）和大小（≤50MB），防止恶意文件上传；
- 数据安全：用户密码通过 BCrypt 加密存储，敏感信息（邮箱）脱敏展示，OSS 文件私有权限控制；
- 接口安全：配置跨域规则，限制允许的请求来源和方法，防止跨域攻击。

10. 如何保证项目的可扩展性和代码可维护性？未来可以从哪些方向扩展？

- 可扩展性设计：按业务域拆分微服务，服务间通过 RESTful API 通信，边界清晰；核心依赖（如大模型、存储）抽象接口，便于替换；
- 可维护性设计：统一代码规范，使用通用响应类和异常处理；通过 AOP 记录操作日志，便于问题排查；Nacos 集中管理配置，支持热更新；
- 未来扩展方向：支持更多文件格式（CSV/CSV）增加 AI 数据预测功能、集成更多模型（如 GPT）添加团队协作功能（文件共享）优化移动端适配。

专属问题

1. 描述一下 Chat2Excel 中 Excel 文件从上传到 AI 分析的完整流程？

- 文件上传：用户通过前端上传 Excel，网关校验 JWT Token，转发至文件服务；
- 文件处理：文件服务校验格式和大小，上传至阿里云 OSS，同时解析 Excel 表头和数据，动态创建 MySQL 数据表，维护文件与表的映射关系；
- AI 分析触发：用户输入自然语言指令，AI 服务接收请求，校验文件权限；
- 意图识别与处理：AI 服务通过提示词判断用户意图（查询 / 修改 / 图表），生成对应的 SQL；

- 结果生成：执行 SQL 查询数据，若为图表生成需求则构造图表数据，通过 SSE 流式推送处理进度和结果；
- 结果反馈：前端展示 AI 分析结论、SQL 语句、图表或修改后的 Excel 下载链接。

2. 项目中如何实现 Excel 文件的解析和动态建表？

- 解析实现：基于 EasyExcel 实现流式读取，避免大文件内存溢出；支持多 Sheet 解析，提取表头作为 MySQL 字段名，统一命名规范；
- 动态建表实现：根据 Excel 表头类型定义 MySQL 字段类型，单文件多 Sheet 对应多张数据表，通过 file_table_mappings 表维护文件与表的关联关系；
- 核心难点：表头格式不统一（如特殊字符、重复表头）大文件解析效率低。

3. AI 服务如何实现自然语言到 SQL 的转化？

- 转化实现：通过 Spring AI 集成大模型，构建标准化提示词（包含文件结构、表头信息、用户指令），引导模型生成符合语法的 SQL；添加 SQL 语法校验和容错机制，避免无效查询；
- 支持的操作：查询类（单表查询、多表关联查询、条件筛选、聚合统计）修改类（数据更新、新增、删除）图表类（柱状图、折线图、扇形图）数据导出类（生成修改后的 Excel 文件）。

4. 流式对话（SSE）的实现原理是什么？

- 实现原理：基于 SseEmitter 建立服务器与客户端的长连接，AI 服务异步处理请求（解析文件→生成 SQL→执行查询→构造结果），分阶段通过 SseEmitter 发送事件（初始化→处理中→完成），客户端接收事件并实时更新界面；
- 长连接处理：设置合理的连接超时时间（300 秒），避免资源浪费；
- 超时处理：在 SSEmitter 创建时设置超时时间，超时后返回友好提示，客户端可重新发起请求；AI 服务优化处理逻辑，提升响应速度，减少超时概率。

5. MCP 服务的核心作用是什么？

- 核心作用：作为“工具适配器”，将现有微服务（用户 / 文件 / AI）封装为符合 MCP 协议的标准工具，提供统一的工具调用接口，简化 AI 模型与外部服务的集成；负责工具注册、协议适配、服务编排和错误处理；
- 支持调用模式：HTTP 模式（网络请求，可远程部署）和 stdio 模式（进程间通信，同一机器部署，适用于本地集成场景）。

6. 网关服务的路由转发和统一鉴权是如何实现的？

- 路由转发实现：基于 Spring Cloud Gateway 配置路由规则，按业务域映射（/api/v1/users/→用户服务、/api/v1/files/→文件服务），通过 StripPrefix 过滤器简化接口路径，实现负载均衡；
- 统一鉴权实现：自定义 JwtAuthGatewayFilterFactory 过滤器，拦截所有请求，从请求头提取 JWT Token，校验有效性（Redis 是否存在、是否过期），有效则解析用户信息传递给下游服务；

- 白名单设计：在 bootstrap.yml 配置无需鉴权的路径（如 /login、/verification-code），过滤器判断请求路径是否在白名单中，若是则跳过鉴权，直接转发请求。

7. JWT 鉴权的完整流程是什么？

- 完整流程：
 - a. 用户登录（验证码 / 密码），服务端验证通过后生成 JWT Token（包含用户 ID / 用户名）；
 - b. Token 存入 Redis 并设置过期时间（默认 20 小时），返回给客户端；
 - c. 客户端后续请求携带 Token 在 Authorization 头（Bearer Token 格式）；
 - d. 网关拦截 Token，校验签名有效性和 Redis 中是否存在；
 - e. 校验通过则解析用户信息，通过请求头传递给下游服务；
- Token 失效处理：Token 过期或不存在时返回 401 状态码，前端跳转登录页；支持刷新 Token 机制，用户操作时若 Token 即将过期，自动生成新 Token 并更新 Redis。

8. 分布式锁在项目是如何应用的？

- 应用场景：Excel 文件上传时，避免同名文件导致的表名冲突；
- 实现方式：基于 Redis 的 setIfAbsent 原子操作，以 “lock_prefix: 文件名” 为 Key，生成唯一值作为 Value，设置过期时间（如 30 秒）；文件服务获取锁后执行建表操作，操作完成后释放锁；
- 解决的问题：多用户同时上传同名 Excel 时，防止并发建表导致的表名重复、数据错乱问题，保证数据一致性。

9. 阿里云 OSS 在项目中起到什么作用？

- 核心作用：存储用户上传的 Excel 文件，解决本地存储扩容和高可用问题；
- 上传处理：文件服务接收 MultipartFile，通过 OSS SDK 将文件上传至指定 Bucket，生成唯一的 OSS Key，存储到数据库；
- 下载处理：用户发起下载请求，文件服务根据文件 ID 查询 OSS Key，通过 OSS SDK 获取文件流，响应给客户端。

10. AI 生成图表的功能是如何实现的？

- 实现流程：
 - a. AI 服务接收用户图表生成指令，识别图表类型和数据维度；
 - b. 生成对应的 SQL 查询数据，处理数据格式（提取 X 轴、Y 轴数据）；
 - c. 构造图表配置信息（图表类型、标题、坐标轴标签）；
 - d. 通过 SSE 流式推送图表数据给前端，前端基于 Chart.js 渲染图表；
- 支持图表类型：柱状图、折线图、扇形图，可根据用户指令扩展其他类型。